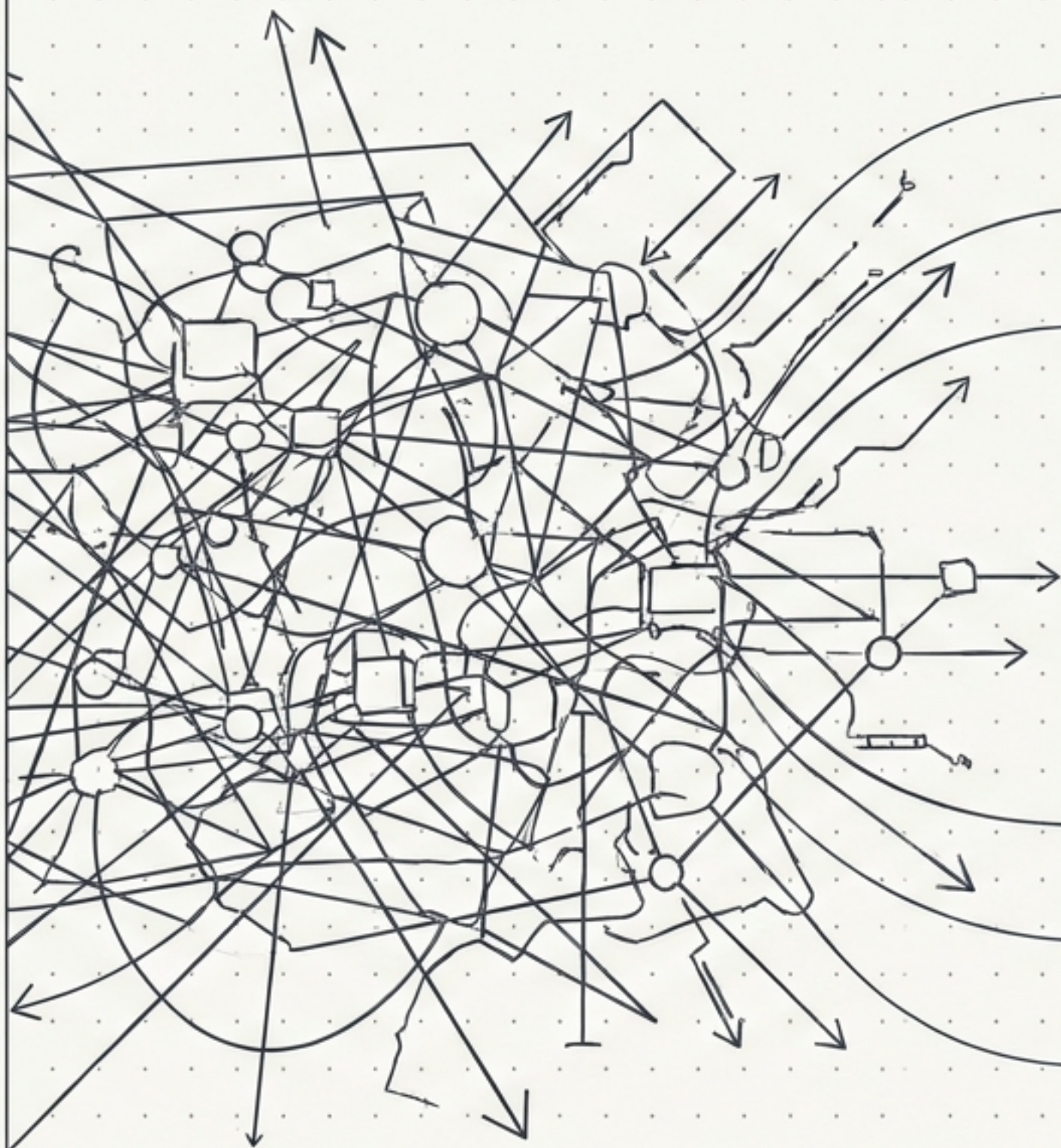




The Python Portfolio Deconstructed

> transitioning from scripts to software

“Authorship isn’t the real test. Understanding is.”

The actual skill demonstrated across these projects is not merely writing code that works, but the engineering judgment behind it—knowing why a structure was chosen, what breaks if it’s reversed, and how to extend it.



Architecture

Decide vs. Act



Resilience

Intentional Error Handling

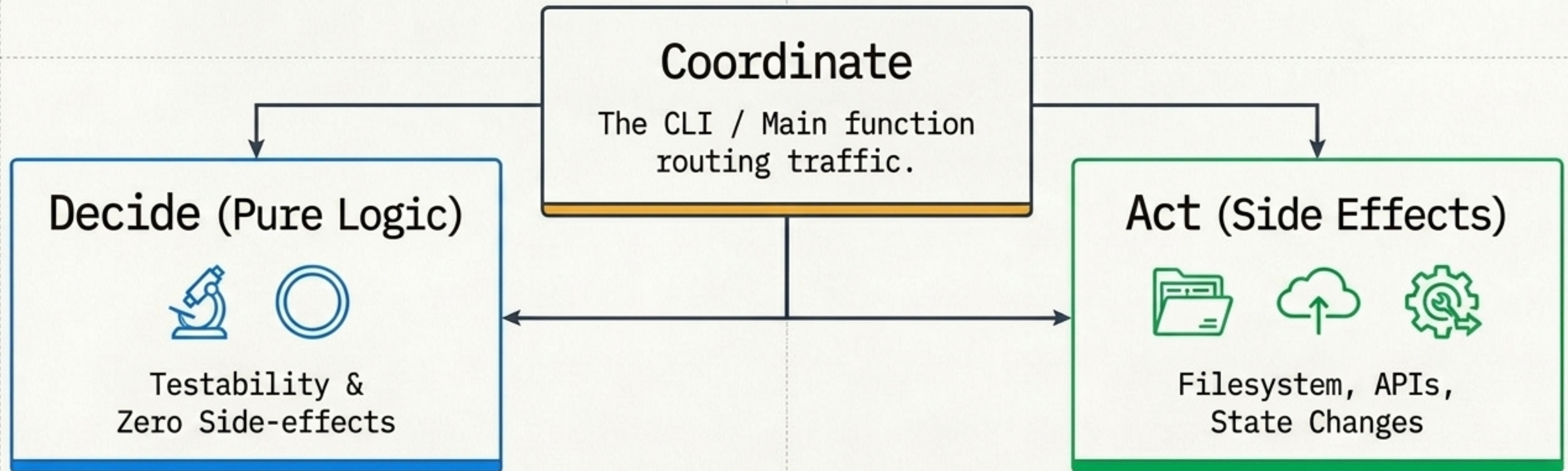


Professionalism

Testing & Standards

The Universal Pattern: Decide vs. Act

Professional software separates the code that makes decisions from the code that touches the outside world. This pattern—whether called domain vs. infrastructure, or **pure functions vs. side effects**—unlocks massive gains in **testability** and **safety**.



Case Study 1: Intelligent File Organizer

Scans a target directory and dynamically sorts files into category subfolders based on their extension. Built entirely with Python's standard library, it features safe collision handling, real-time logging, and a zero-risk preview mode.

Before: Chaos

/Target_Directory/

- photo.jpg
- budget.csv
- script.py
- song.mp3



After: Categorized

/Target_Directory/

[Images]

photo.jpg

[Documents]

budget.csv

[Code]

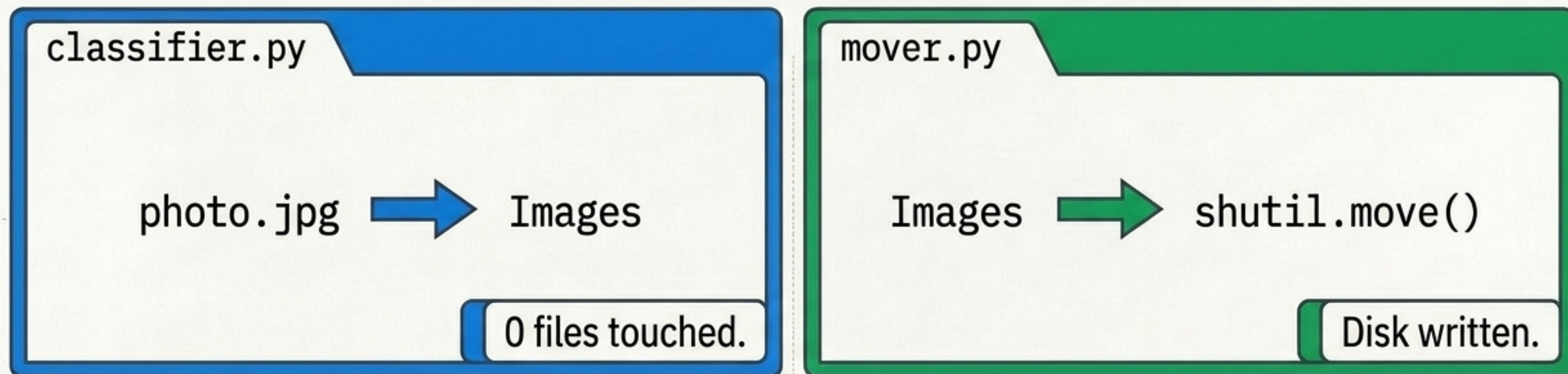
script.py

[Audio]

song.mp3

Architecture: Decoupling Logic from I/O

The classifier never touches a real file; it only answers a question. The mover executes the action.



Why? Free Dry-Runs.

By passing a `dry_run` flag to a single function, we skip only the disk write.

The preview logic is guaranteed to stay perfectly in sync with the execution logic.

Project 1 Toolkit: Modern Filesystem Management

Card 1

`pathlib.Path`

Object-oriented paths.
`Path('file.txt').suffix` replaces
brittle string parsing.

Card 2 `dict.get(key, default)`

Safe dictionary lookups. Unknown
extensions fall back to 'Other' instead of
crashing with a `KeyError`.

`shutil.move()` & `mkdir(exist_ok=True)`

Idempotent folder creation and cross-drive
safe moving (unlike OS-level renames).

Card 4 `argparse` & `logging`

CLI flag parsing combined with simultaneous
console and timestamped file audit trails.

Stress Test: Edge Cases & Trade-Offs

Panel 1

⚠️ Issue | ⚠️ The Collision Threat

Two files named data.csv.

✅ Solution

`resolve_destination()` dynamically appends counters (`data_1.csv`) to prevent irreversible data loss.

Panel 2

⚠️ Issue | ⚠️ The Recursion Trap

Infinite loops into newly created category folders.

✅ Solution

Intentionally shallow scanning (`target_root.iterdir()`) restricted to one level deep.

Panel 3

⚠️ Issue | Targeted Exceptions

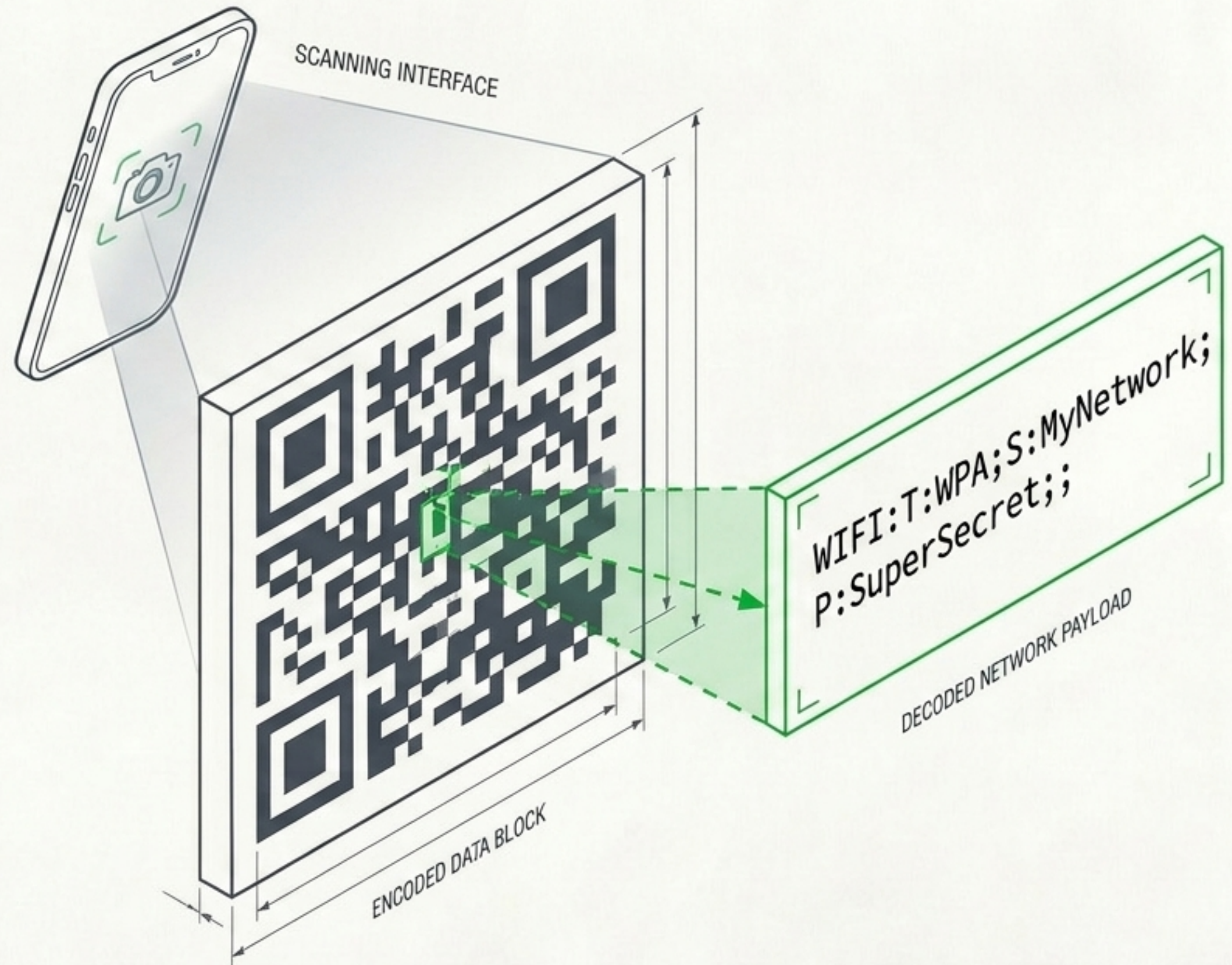
Hiding real bugs with bare `except: blocks`.

✅ Solution

Catching expected `PermissionError` (locked files) to skip/log, while allowing `NameError` (developer bugs) to crash loudly.

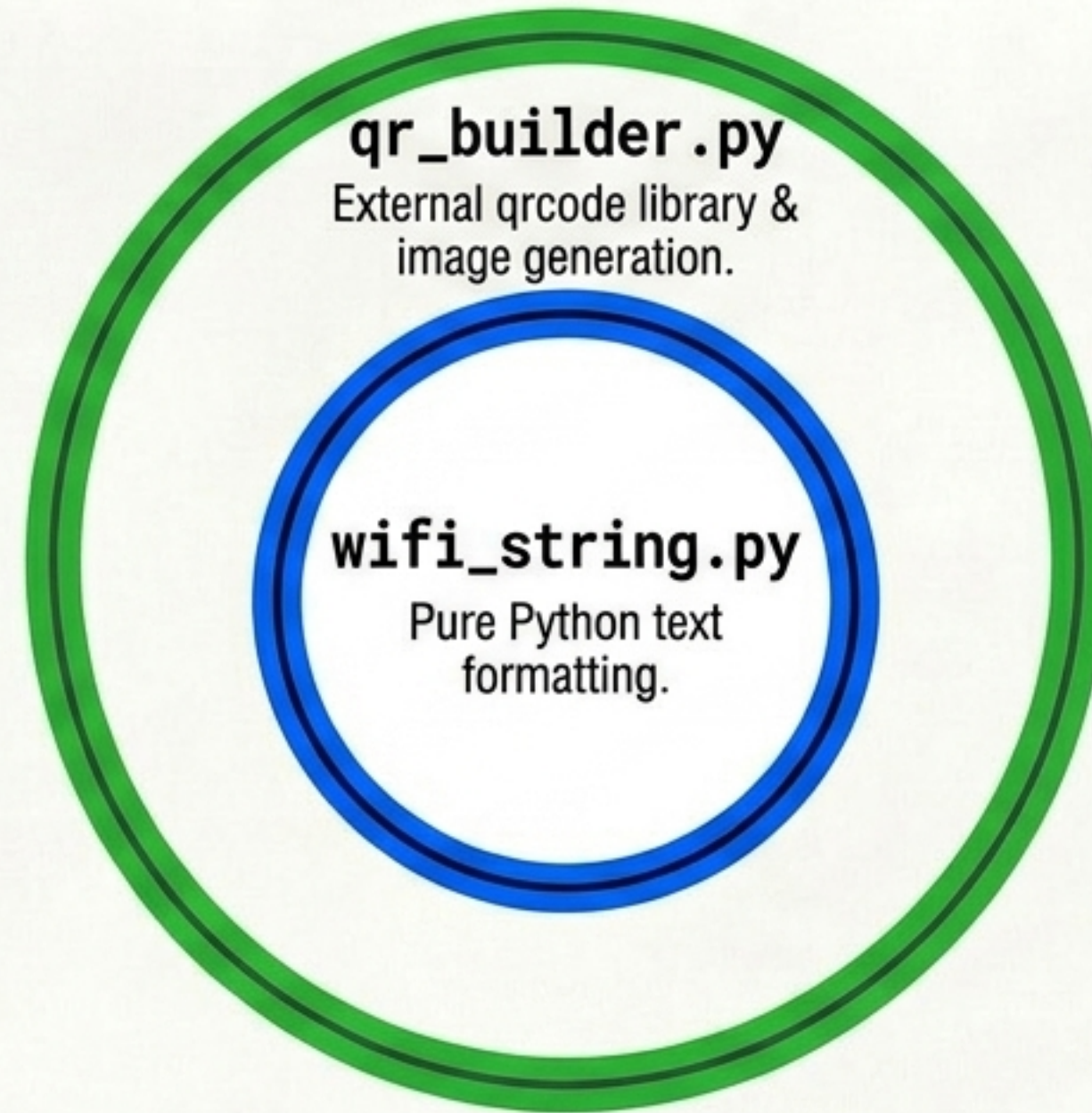
Case Study 2: WiFi QR Generator

Takes network credentials (SSID, password, encryption type) and compiles them into a scannable PNG. Phone cameras natively recognize this de facto standard text string and trigger an auto-connect prompt without manual typing.



Architecture: Isolating External Dependencies

The 'Decide vs. Act'
pattern applies here
by isolating
third-party risk.



`wifi_string.py` builds and validates the exact text format phones expect. Because it has zero image logic, the core business rules can be rigorously tested without even having the external image libraries installed.

Project 2 Toolkit: Standards & Third-Party Code

The Format (Error Correction)

Utilizing `Level M redundancy` to recover from `15% data loss`—a calculated middle ground between compactness and damage resilience.

Dependency Pinning (`requirements.txt`)

Explaining the difference between exact pinning for direct dependencies (`qrcode==8.2`) and minimum floors for transitive dependencies (`pillow>=9.1.0`) to avoid `brittleness`.

Free Validation (`argparse`)

Using `choices=['WPA', 'WEP', 'nopass']` to `reject invalid input` at the CLI layer before the program's `business logic` even executes.

Stress Test: Data Integrity & Testing

Manual Character Escaping

Why not use a library?

The 4-character rule (escaping `;;` with a backslash) is simple. A manual replace is highly efficient and avoids pulling in a heavy, general-purpose escaping dependency.

Missing Data Prevention

Rejecting WPA requests that lack passwords immediately, rather than passing empty data through and generating a misleading QR code.

Testing without I/O

Utilizing Python's `tempfile` module and `tearDown()` in tests to generate images in ephemeral directories, ensuring tests never pollute the real filesystem with ghost images.

The Architecture of Judgment: When to Guess vs. When to Crash

	Project 1 (File Organizer)	Project 2 (WiFi QR)
The Scenario	Unknown file extension.	Empty SSID or invalid encryption.
The Action	Fallback to 'Other' (Fail Gracefully).	Raise <code>ValueError</code> immediately (Fail Loudly).
The Rationale	The cost of being wrong is low (user manually moves a file).	The cost of being wrong is high (silently generating a broken QR code that frustrates users with no explanation).

The Professional Baseline

Blueprint Legend	
<code>if __name__ == '__main__':</code>	Guarantees module safety, preventing execution when imported elsewhere.
<code>def func() -> str:</code>	Type hints improve developer UX and enable static analysis (mypy).
<code>"""Google-style"""</code>	Standardized docstrings ensure tooling compatibility and readable contracts.
<code>unittest</code>	Validating pure logic with isolated state before a user ever touches the application.
<code>.gitignore</code>	Repository hygiene; preventing OS junk and local logs from polluting version control.

Explain the Why.

The code is merely the output; engineering judgment is the skill.
By separating concerns, managing dependencies intentionally, and
tailoring error handling to the user's context, a
developer proves readiness for production environments.

These projects demonstrate not just how to build, but how to design software that scales, adapts, and endures.